Can a Language Model Write the Weights?

An Empirical Study of LLM-Written Scoring Networks for Column Generation and MILP Branching, with a Bus-Driver Scheduling Application

Renan Butkeraites
BIS — Butkeraites Intelligent Solutions
renan@butkeraites.com

November 2026

Abstract

Weight-space programming (WSP) is the idea that a large language model (LLM) can write the weights of a small neural network directly, guided only by reading the outputs of forward passes, with no gradient training [Takehi et al., 2026]. We test whether WSP can improve classical, exact operations research algorithms in places where they rely on cheap scoring rules. In four studies, an LLM wrote small multilayer perceptrons that (i) select arcs, labels and ng-neighbourhoods inside column generation for the vehicle routing problem with time windows, and (ii) choose branching variables in SCIP's branch-and-bound. The algorithms stay exact whatever the weights are; a bad network only costs time. Results: in column generation, no lever gave a material gain (exact pricing dominates the running time). In branching on synthetic benchmark classes, a hand-written network solved independent-set instances in 0.37× the time of SCIP's default rule (18 paired instances, Wilcoxon $p < 10^{-5}$) and matched or beat networks trained by imitation of strong branching on the same features. The same rule was not general: on 39 MIPLIB 2017 instances it solved 19 versus 24. In an application built from the real GTFS feed of Atlanta's transit agency (MARTA), a crew scheduling set-partitioning model solved by a native C plugin with SCIP's factory settings, the LLM-written rules took 0.63× the default's time on 32 held-out instances ($p < 10^{-4}$) — but SCIP's own plain pseudocost rule did exactly as well (ratio 0.99, $p = 0.19$). We conclude that, in these settings, the gains came from being cheap, not from what the LLM wrote. All code, logs and the numbers behind every figure are released.

AI-generation and provenance statement (read first). This paper — its text, figures, tables, code, experimental design and analysis — was generated by an AI model, **Claude Opus 5.5 (Anthropic)**, running as an agent in Claude Code, under the direction of the human author. The human author chose the research questions, approved or redirected each step, and takes full responsibility for the content; the AI is not an author. **All numerical results are from computations actually run** on the author's local machine (Apple M5, 10 cores, 32 GB RAM, macOS 27) between 2026-11-03 and 2026-11-05; no number was estimated, simulated or typed in by the model. Every number in the paper is regenerated from the raw run logs by one script (`scripts/figs.py`, Appendix A). Every reference was checked against the publisher's page, a DOI resolver, dblp or the original web page. Note the circularity: the same model that wrote this paper also wrote the network weights that are the object of study. Appendix B lists what the model did and what the human did.

1 Introduction

Exact algorithms in operations research (OR) contain many small decisions that are made by cheap scoring rules: which arcs to keep in a pricing graph, which labels to extend, which fractional

variable to branch on. These rules are where machine learning has entered the field [Bengio et al., 2021]: branching rules have been learned by ranking [Khalil et al., 2016], by regression on strong-branching scores [Marcos Alvarez et al., 2017] and by graph neural networks trained to imitate strong branching [Gasse et al., 2019]. All of them need training data and a training pipeline.

A recent post, "Sparks of Weight-Space Programming" [Takehi et al., 2026], reports that an LLM can instead *write* the weights of a small network by hand, reading the outputs of forward passes and editing the weights, without any gradient step. If that works for OR scoring rules, it would give a third option between hand-coded heuristics and trained models: a network whose every weight has a written justification, specialised to a problem family in hours, with no training set.

This paper asks a narrow, testable question:

> *Can an LLM write the weights of a cheap scoring network that makes a gold-standard exact OR algorithm faster, on held-out instances, without changing its result?*

We ran four studies (Figure 1). The setting is deliberately favourable to measurement: the host algorithms are exact, so the network can only change time and tree size, never the optimum; and we always compare against the incumbent expert rule in the same batch of runs.

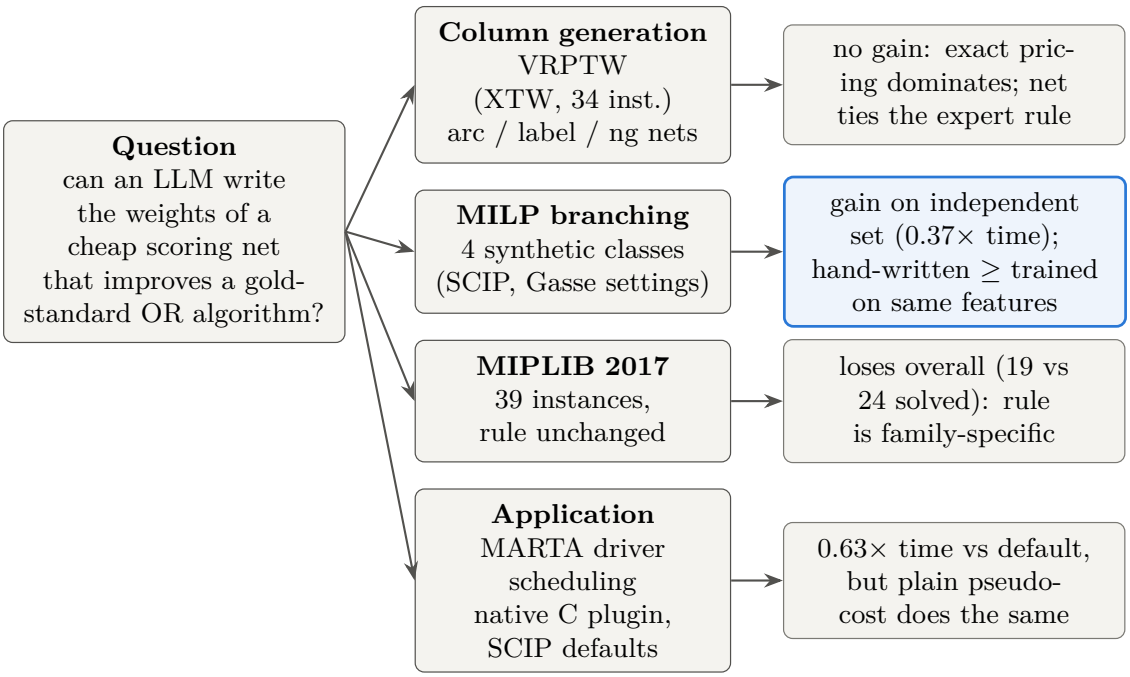


Figure 1: Map of the study. Each row is one study; the right column is its verdict. Only the synthetic branching study (blue) found a gain attributable to the written weights, and it did not transfer to MIPLIB or to the application.

Findings at a glance. Each finding has an identifier used in the claims table (Appendix C), which points to the figure, table and log file that support it.

F1. Column generation for VRPTW: no material gain. Exact pricing takes most of the time, and the LLM-written arc scorer ties the classical reduced-cost rule (time ratio 1.02 over 34 instances; Section 4).

F2. Synthetic MILP branching: on independent set, the LLM-written rule needs 0.37× the time of SCIP's default with similar or smaller trees ($n = 18$, $p < 10^{-5}$); on set cover it ties (0.90×, $p = 0.14$) with larger trees (Section 5).

F3. With the same 7 features, hand-written weights match or beat weights trained by imitation of strong branching in 3 of 4 settings, although the trained weights imitate strong branching better (Section 5.1).

F4. The rule is family-specific: on 39 MIPLIB 2017 instances it solves 19 vs 24 (Section 6).

F5. Application (MARTA bus-driver scheduling, native C plugin, SCIP defaults, 32 held-out instances): LLM-written rules take 0.63× the default's time, but SCIP's plain pseudocost rule achieves the same (0.99× relative to it, $p = 0.19$). The gain is "skip strong branching", not WSP (Section 7).

F6. A net-guided lookahead (strong branching only on the network's top-k) halved tree sizes relative to SCIP's default on 8 practice instances, but did not generalise to the test set (1.05× time, $p = 0.51$).

2 Background

Column generation. Column generation solves a linear relaxation with exponentially many columns by alternating a restricted master LP and a pricing problem that finds columns with negative reduced cost [Lübbecke and Desrosiers, 2005, Desaulniers et al., 2005]. For the vehicle routing problem with time windows (VRPTW) the pricing problem is an elementary shortest path problem with resource constraints [Desrochers et al., 1992], commonly relaxed to ng-routes [Baldacci et al., 2011]. Practical codes price heuristically on a reduced graph first and fall back to exact pricing; dual smoothing [Wentges, 1997] stabilises the duals.

Branching. In LP-based branch-and-bound, the choice of the branching variable drives tree size. Full strong branching solves the two child LPs for every candidate and picks the best product of gains; it gives small trees at a high cost per node. Reliability pseudocost branching, SCIP's default, uses strong branching only until pseudocost estimates are reliable [Achterberg et al., 2005, Achterberg, 2007]. Learning to branch tries to approximate strong branching cheaply [Khalil et al., 2016, Marcos Alvarez et al., 2017, Gasse et al., 2019]. We use SCIP [Achterberg, 2009, Hojny et al., 2025] through PySCIPOpt [Maher et al., 2016] and natively in C.

Weight-space programming. In WSP [Takehi et al., 2026] the LLM is the optimiser: it proposes weights, observes forward-pass outputs, and edits the weights in text. Our protocol (Figure 2) follows that spirit: diagnostics are *read* by the model and never fitted.

3 Method

Where the network sits. In branching (Figure 3), at every node SCIP calls our branching rule with the fractional candidates. We compute a feature vector per candidate from quantities already available at the node (Table 1), evaluate the network, and branch on the arg max. The network is a multilayer perceptron stored as JSON; most networks in this paper have one linear layer or one ReLU layer of width 6. An optional *lookahead* runs strong branching only on the network's top-k candidates, and turns any child proven infeasible into a free bound change, as SCIP's own strong branching does.

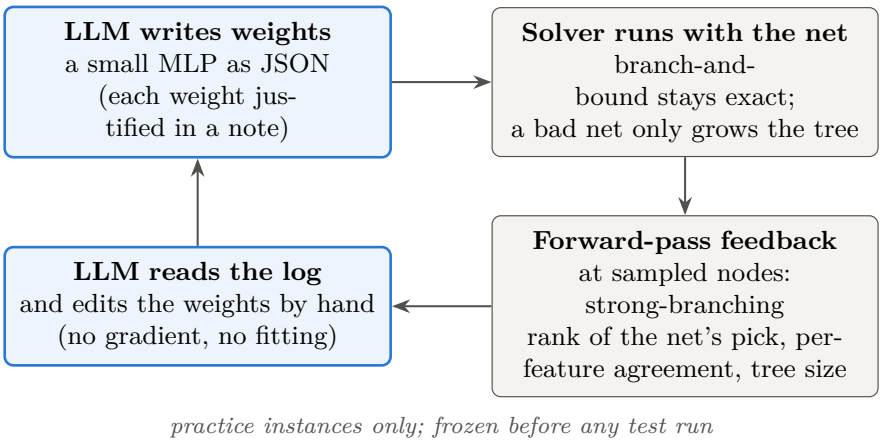


Figure 2: The WSP protocol used in all studies. Weights are written by the LLM as a JSON file with a free-text note justifying each weight; feedback runs are read, never fitted; the network is frozen before the held-out test.

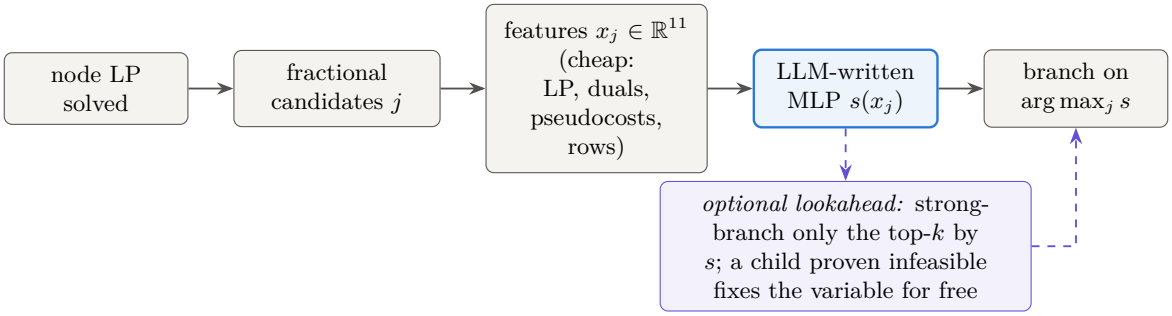


Figure 3: The branching rule at one node. The dashed path is the optional net-guided lookahead (Section 7).

Table 1: Candidate features. Features marked † are divided by their maximum over the node's candidates. The last four were added for set-partitioning structure in the application.

Feature	Definition (candidate j with LP value v_j, fractional part f_j)
`frac`	$2\min(f_j, 1-f_j)$; 1 means exactly one half
`ps`†	SCIP pseudocost score of j
`obj`†	$\lvert c_j \rvert$
`deg`†	number of LP rows in which column j has a nonzero
`tight`	share of j's rows that are tight at the LP optimum
`dualm`†	$\sum_i \lvert y_i a_{ij} \rvert$ with y the row duals
`val`	v_j
`fconf`†	LP mass of the other fractional candidates sharing a row with j
`alt`	$1/(m-1)$, m the fewest fractional candidates in any row of j
`rowf`†	number of j's rows touched by more than one fractional candidate
`eff`	cost per row of the most efficient candidate divided by that of j

Protocol rules. (1) Separate practice and test instance sets generated with different seeds; weights are written on practice only and frozen before the test. (2) Every comparison is *paired*: all rules run on the same instances in the same batch, so they share machine load. (3) Tree size (nodes) is deterministic for a given SCIP build and seed; time is noisy across batches, so time is only compared within a batch. (4) Summary statistics are shifted geometric means (shift 10 for nodes, 1 s for time) with time-outs counted at the limit. (5) Paired two-sided Wilcoxon signed-rank tests on log time ratios; pairs that both hit the time limit are excluded from the test.

Software. SCIP 10.0 through PySCIPOpt 6.3.0 for Studies 2–3 and the Python prototype of Study 4; SCIP 10.1.0 (SoPlex 8.1.0) linked from C for the final application runs; HiGHS [Huangfu and Hall, 2018] through the `highs` Rust crate 2.4.0 for column generation; Python 3.12, Rust 1.96. Studies 2–3 use the settings of Gasse et al. [2019] (no restarts, cuts only at the root, fixed seed); the final application runs use SCIP's factory settings.

4 Study 1: column generation for VRPTW

Setup. We implemented column generation for VRPTW in Rust: a set-covering master LP kept warm in HiGHS, ng-route labelling with ng-sets of size 8, Wentges smoothing, heuristic pricing on a reduced graph with a label cap, and exact pricing as fallback, so the final bound is exact for every strategy. Instances are the XTW set released on CVRPLIB [Uchoa et al., 2026]; 34 instances with up to 256 customers for the main comparison and the 10 largest (819–1001 customers) for a scale test. The LLM wrote networks for three levers: (a) arc scoring for the reduced graph, compared with the classical rule "keep the K arcs with lowest reduced cost per customer"; (b) label eviction under the cap; (c) ng-neighbourhood selection.

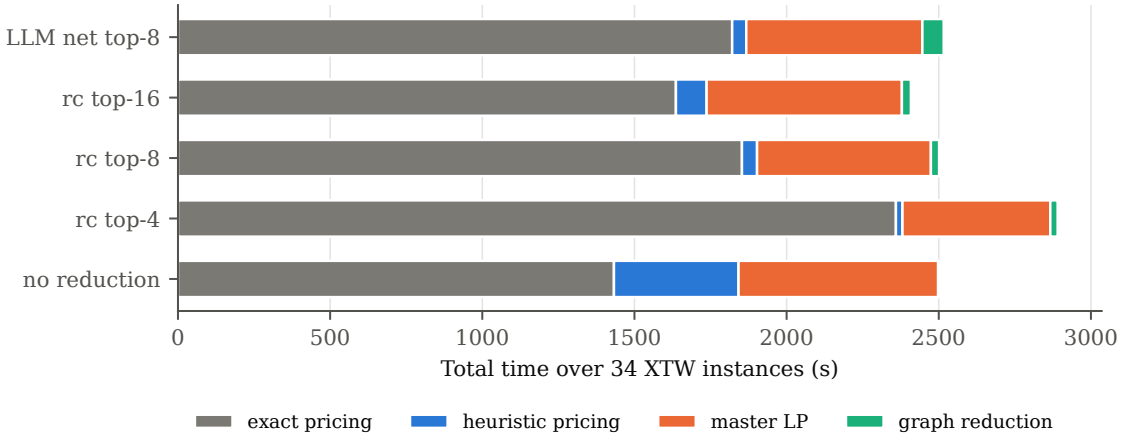


Figure 4: Where column-generation time goes, summed over 34 XTW instances, for each arc-reduction strategy. Exact pricing dominates in every strategy, so better heuristic arcs can save little; the LLM-written net (top) ties the reduced-cost top-8 rule.

Results. Figure 4 shows the time breakdown. The arc network ties the reduced-cost top-8 rule (geometric mean time ratio 1.02). Label-eviction networks did not beat "keep the cheapest labels", and raising the label cap from 8 to 64 cut exact-pricing time by only about 13% while multiplying

heuristic time about tenfold, so the eviction policy had little headroom. ng-set selection that drops neighbours unable to close a cycle raised the bound by 0.03–0.15% on average; a replicated timing at ng-8 gave 0.93× time, within noise. On the largest instances, in heuristic-only mode, the arc network reached a lower master objective than the top-8 rule at equal iteration on 7 of 10 instances, but its unoptimised scoring cost 40–197 s, so it lost at equal time. **Verdict (F1):** no material gain.

5 Study 2: branching on synthetic classes

Instances. We generated the four classes used by Gasse et al. [2019]: set cover [Balas and Ho, 1980] (1000 rows × 1000 columns, density 0.05), combinatorial auction [Leyton-Brown et al., 2000] (100 items, 1500 bids), capacitated facility location [Cornuéjols et al., 1991] (70 customers, 70 facilities) and maximum independent set (750 nodes, affinity 4) with a clique-partition formulation, following the procedure that Gasse et al. [2019] attribute to Bergman et al. [2016]; like their public code, we draw the graphs from the Barabási–Albert model [Barabási and Albert, 1999]. Practice: 4 per class; test: 10 per class; medium (larger, new seed): 8 per class for set cover (1250 rows) and independent set (1000 nodes); a separate training set of 10 per class for the trained baseline. Time limit 300 s.

Headroom first. Before writing any weights we measured the gap between SCIP's default and full strong branching (Figure 5). On set cover and independent set, strong branching trees are 26× and 19× smaller than the default's but about 4–5× slower, which is the regime where a cheap, well-aimed rule can pay off. Facility location and auctions have trees of tens of nodes and little headroom.

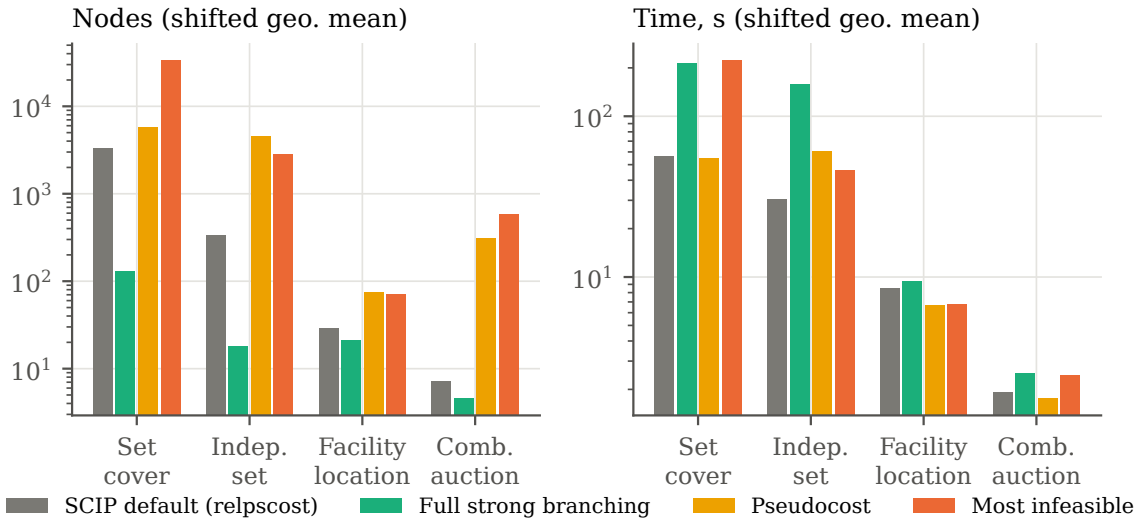


Figure 5: Headroom on the synthetic test set (10 instances per class, 300 s). Full strong branching builds far smaller trees on set cover and independent set, at a much higher cost per node.

The written networks. The LLM read per-feature agreement with strong branching on the practice set and wrote network v0, then edited it into v1 after reading where v0's picks disagreed

with strong branching. In closed form, v1 scores a candidate as

$$s_{\mathrm{v1}}(x) = \texttt{deg} + 0.7\,\texttt{dualm} + 0.4\,\texttt{val} + 0.2\,\texttt{ps} + 0.3\,\texttt{frac} - 3\,\max(0,\ 0.4 - \texttt{frac}). \tag{1}$$

The note stored with the weights explains each term: `deg` favours hub variables whose up-branch propagates widely; `dualm` measures how much LP dual pressure the column carries; the ReLU gate removes nearly integral candidates; the pseudocost weight was cut from 0.7 to 0.2 because the disagreements came from candidates with high but noisy early pseudocosts.

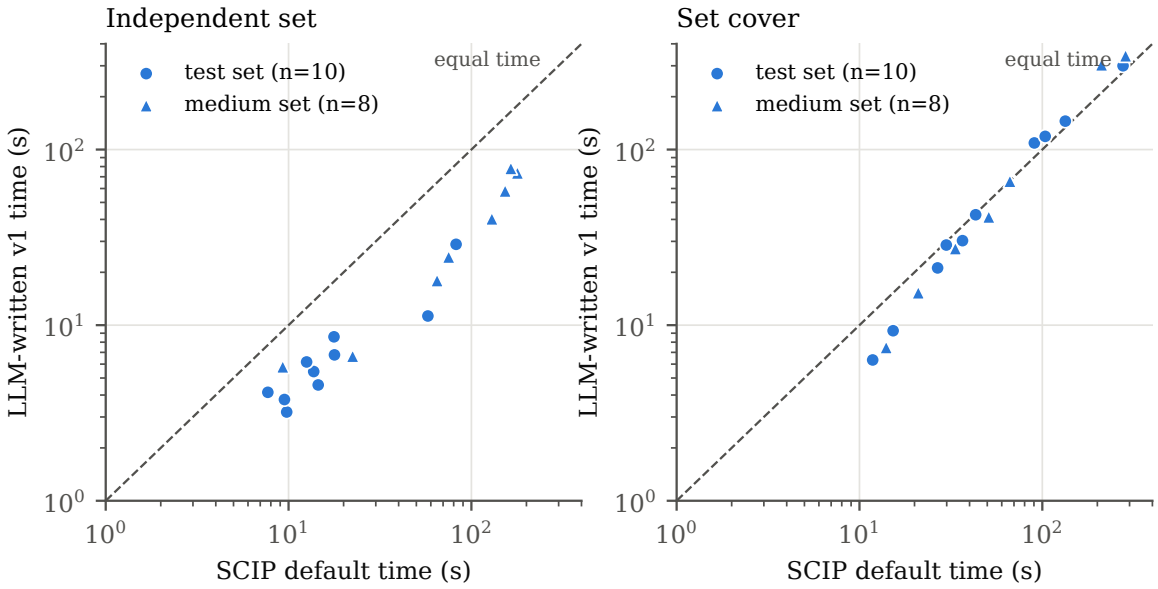


Figure 6: Per-instance time, LLM-written v1 versus SCIP default, paired runs, test and medium sets. Points below the diagonal are instances where v1 is faster. Independent set: 18 of 18 faster.

Results. Figure 6. On independent set, v1 is faster on all 18 instances, with a geometric-mean time ratio of 0.37 ($p = 7.6 \times 10^{-6}$) and smaller trees on the medium set (1426 vs 2786 nodes). On set cover it ties on time (0.90, $p = 0.14$) with 1.3–1.4× larger trees. The network's features are computed in Python while SCIP's rules run in C, so these time ratios are pessimistic for the network. Optimal values were identical on every instance solved by both rules. **(F2)**

5.1 Hand-written versus trained weights

To separate "LLM-written" from "a network on these features", we trained the same architecture (7 inputs, 16 ReLU units) by imitation of strong branching: listwise softmax cross-entropy with the strong-branching best candidate as target, Adam, 20% of instances held out, on 2,166 labelled nodes (1,361 set cover, 805 independent set) from the separate training set. Figure 7 shows imitation quality; Figure 8 shows the paired solving results.

The trained networks imitate strong branching better on independent set (0.44–0.46 vs 0.39), yet produce larger trees in 3 of 4 settings and are never faster than v1. Better imitation did not mean better decisions. **(F3)** This is a deliberately small baseline (same 7 features, 2k nodes), not a reproduction of the graph network of Gasse et al. [2019].

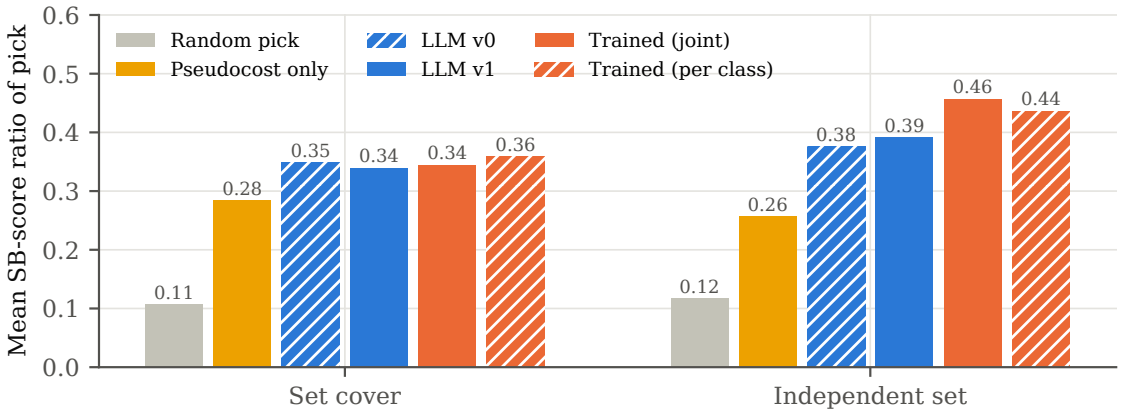


Figure 7: Imitation quality on the training nodes: mean ratio between the strong-branching score of the picked candidate and the best one (1 = always picks the strong-branching best). Trained networks imitate strong branching better on independent set.

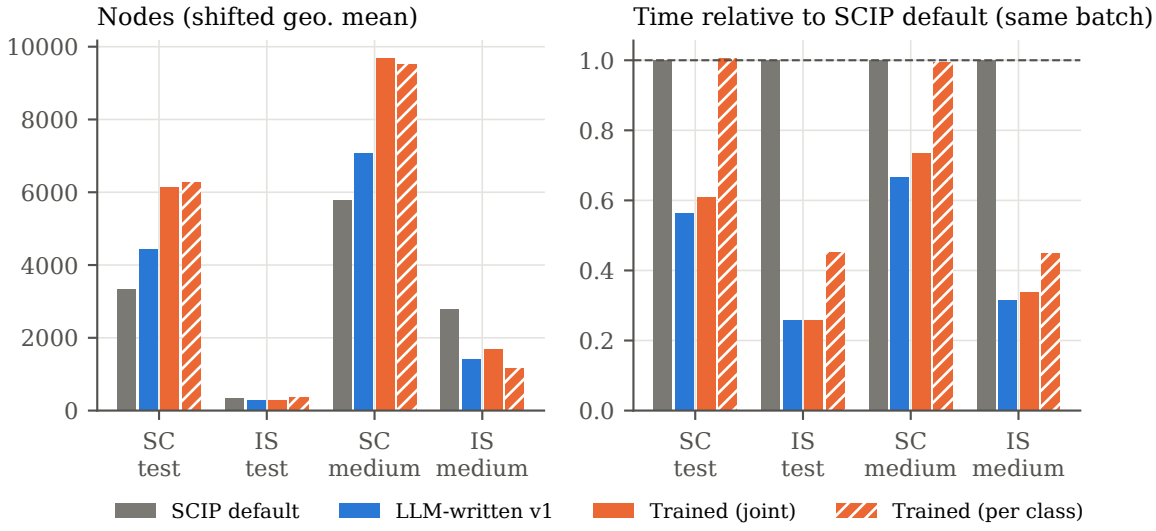


Figure 8: Hand-written v1 versus trained networks with identical features, paired batch. Left: tree size. Right: time relative to SCIP's default in the same batch. SC = set cover, IS = independent set.

6 Study 3: generalisation to MIPLIB 2017

We ran v1, unchanged, on 40 instances of MIPLIB 2017 [Gleixner et al., 2021] chosen from the benchmark set to be solvable within minutes (one could not be read), paired with SCIP's default, 300 s, same settings. Figure 9 shows per-instance time ratios.

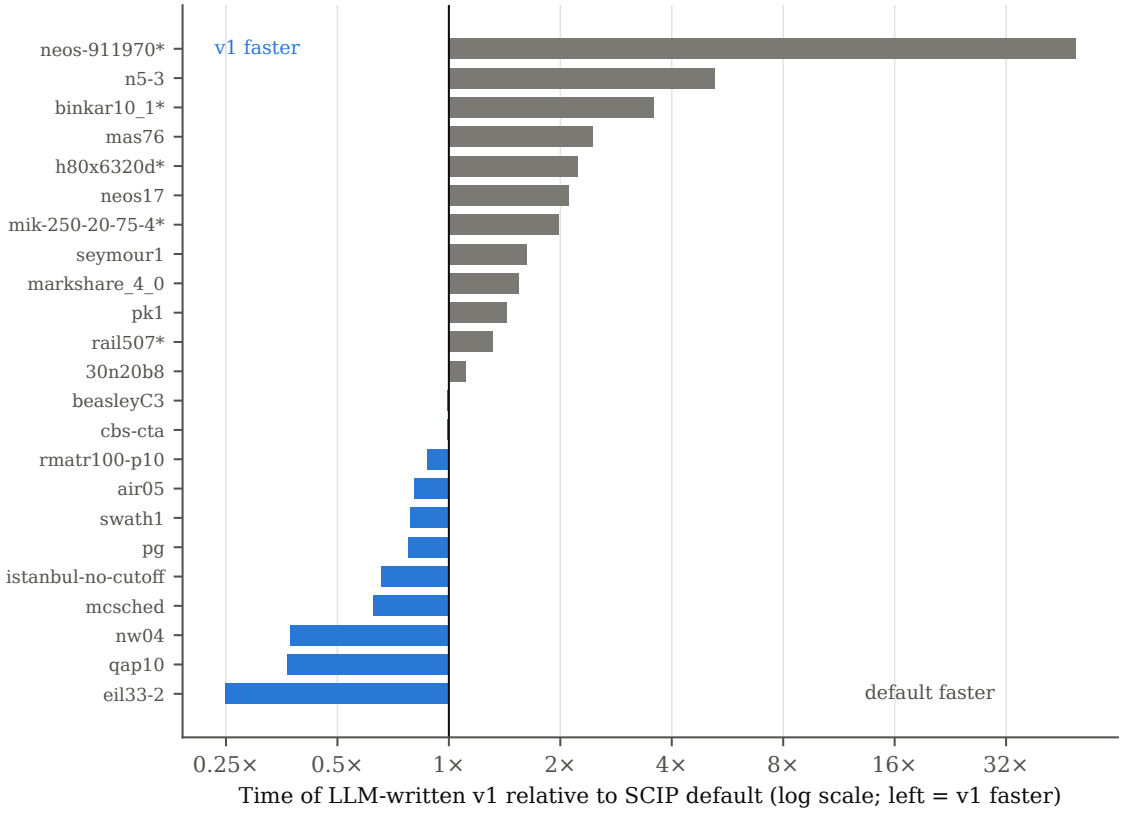


Figure 9: MIPLIB 2017: time of v1 relative to SCIP's default, per instance (instances where both rules hit the time limit, or both finish within 10 s, are omitted). * = at least one rule hit the 300 s limit.

v1 solves 19 instances against 24 for the default; on the 17 non-trivial instances solved by both, its trees are 2.6× larger (5080 vs 1989 nodes). It wins clearly on airline crew scheduling (`nw04`), a quadratic assignment instance (`qap10`) and a routing instance (`eil33-2`), and loses badly on others (`neos-911970`: 6 s for the default, time-out for v1). A rule written for two synthetic families is not a general-purpose rule. **(F4)**

7 Study 4: application to bus-driver scheduling (MARTA)

The pitch for WSP in practice is per-family specialisation: a company that solves the same model every day could have a rule written for its family. We tested this on driver scheduling (run-cutting), a classic transit problem [Wren and Rousseau, 1995] usually written as set partitioning [Ryan and Foster, 1981].

Instances from real data. From the public GTFS feed of the Metropolitan Atlanta Rapid Transit Authority [Metropolitan Atlanta Rapid Transit Authority (MARTA), 2026] we took every bus trip with its vehicle block, times and terminals (6,918 weekday bus trips on 81 routes). A "garage" is a set of three routes whose blocks stay within them (GTFS does not publish garage assignment). Within each block, a *piece* is a run of consecutive trips; relief (driver change) happens at trip boundaries. A duty is one piece of 5–8.5 h of work (straight), two pieces with a 30–90 min meal break or a 1.5–5 h split break (spread $\leq$ 12 h, relief within 400 m), or a short "tripper" piece with a penalty (Figure 10). Costs: a fixed cost per driver, paid minutes with an 8 h guarantee, and split and tripper penalties. At most 15 second pieces are sampled per first piece and instances above 12,000 duties are discarded. The labour rules and the garage grouping are synthetic; trips, blocks, times and terminals are real. Practice: 8 non-trivial garages from a pool of 40 (seed 7); test: 32 new garages (seed 11), weekday, alternate weekday, Saturday and Sunday service, used without any filtering.

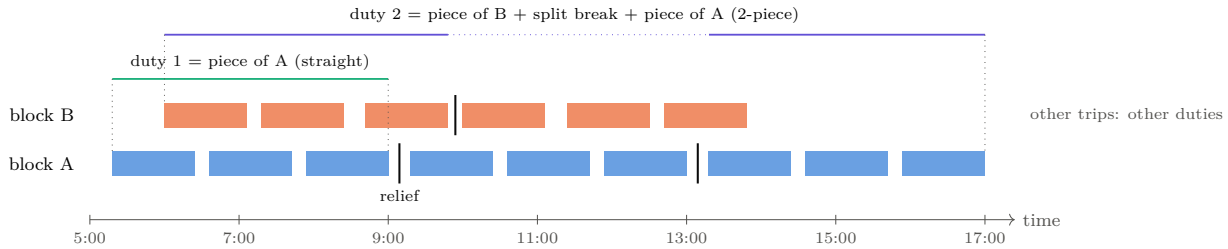
Headroom and diagnosis. Strong branching solved practice garages with 11–75 nodes against 286–2401 for the default (SCIP 10.0, Gasse settings). The per-feature diagnostics (Figure 11) showed that no generic feature is informative and that in 91% of nodes some candidate has a child that strong branching proves infeasible or worse than the incumbent: the good decision is "which variable can be fixed for free", which cheap features cannot see. The LLM added the four set-partitioning features of Table 1; they were no better.

Forward passes and the lookahead. The LLM wrote four linear candidates and measured tree size and time on practice; the best (`m3`: `frac` + 0.5 `ps` + 0.3 `val` + 0.3 `alt` + 0.5 `rowf`) was then combined with a net-guided lookahead (Figure 3). To make time comparable with SCIP's C rules, the rule was ported to a native SCIP plugin in C (`wspsolve.c`); the C port reproduces the Python node counts (1226 vs about 1290 on one garage). On practice, with factory settings (Figure 12), strong branching on the network's top-3 with a 100-iteration limit produced trees 0.56× the default's and ran in 0.81× its time; top-8 gave 0.40× the nodes.

Held-out test. Figure 13 and Table 2. Against SCIP's default, the LLM-written rules without lookahead are faster on about 70% of garages, with a geometric-mean time ratio of 0.63 ($p = 4 \times 10^{-5}$), and solve as many or more instances. The lookahead variant did not generalise (1.05×, $p = 0.51$, 22 solved). The decisive control is SCIP's own pseudocost rule, which uses no strong branching at all: it is as fast as the LLM-written rules (`vm_fast`/`pscost` time ratio 0.99, $p = 0.19$), and the MARTA-specific network is no better than the generic v1 (ratio 1.00, $p = 0.61$). **(F5, F6)**

Table 2: MARTA held-out test, native C, SCIP 10.1 factory settings, 300 s, paired. Shifted geometric means over all 32 garages (time-outs at the limit). The first four rows share one batch; the last two rows come from a second batch with its own default run.

Rule	Solved	Nodes	Time (s)
SCIP default (relpscost)	24	798	88.9
SCIP pseudocost	24	2074	57.2
LLM v1 (generic, from Study 2)	25	1987	56.9
LLM `vm_fast` (MARTA)	24	1980	56.6
SCIP default (second batch)	23	782	88.3
LLM `vm_la` (net + top-3 lookahead)	22	640	93.8



Set partitioning (run-cutting): $\min \sum_d c_d\, y_d$ s.t. $\sum_{d \ni t} y_d = 1 \;\; \forall$ trips t, $y_d \in \{0,1\}$, c_d = fixed driver cost + paid minutes (+ split / tripper penalties).
Rows, blocks, times and terminals come from MARTA's public GTFS; labour rules and garage grouping are synthetic.

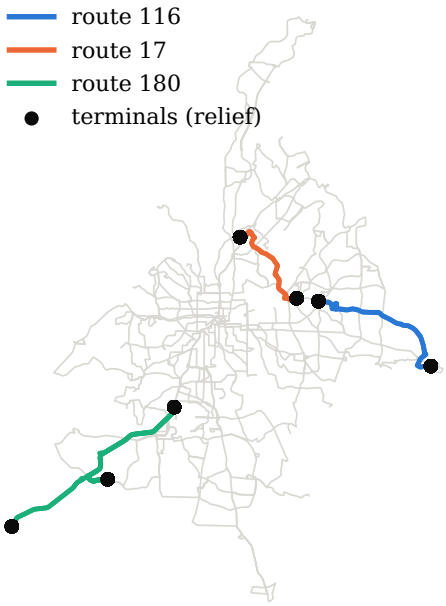


Figure 10: Top: from vehicle blocks to duties to the set-partitioning model. Bottom: one practice garage (three MARTA bus routes, coloured) over the whole bus network (grey); dots are terminals where relief can happen.

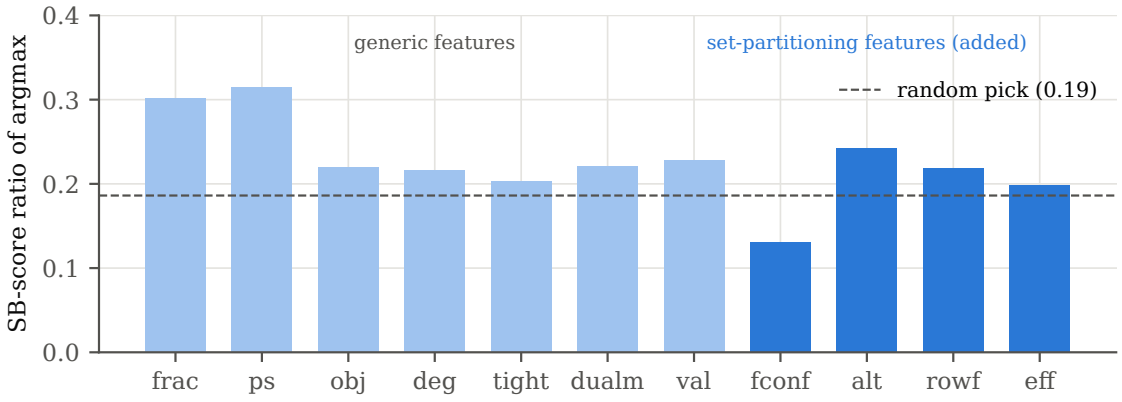


Figure 11: MARTA practice diagnostics (2,264 nodes): strong-branching score ratio obtained by branching on the arg max of each single feature. The dashed line is a random pick.

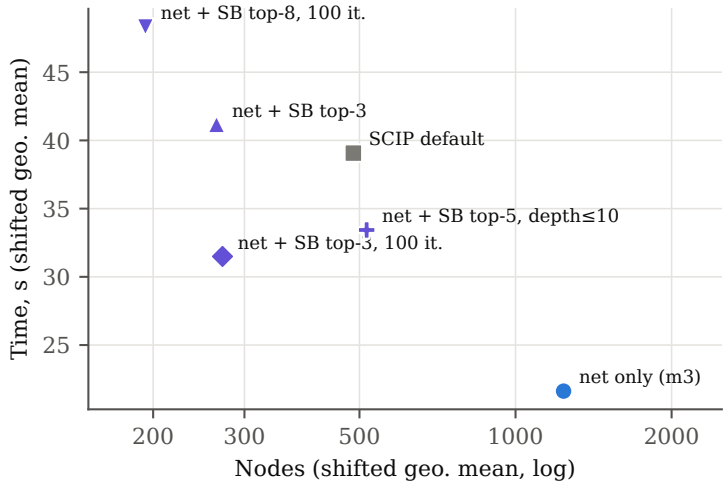


Figure 12: MARTA practice set (8 garages, native C, SCIP factory settings): tree size versus time for SCIP's default, the network alone, and network-guided lookahead variants. Two rules were frozen from this plot: `vm_fast` (network only) and `vm_la` (top-3, 100 iterations).

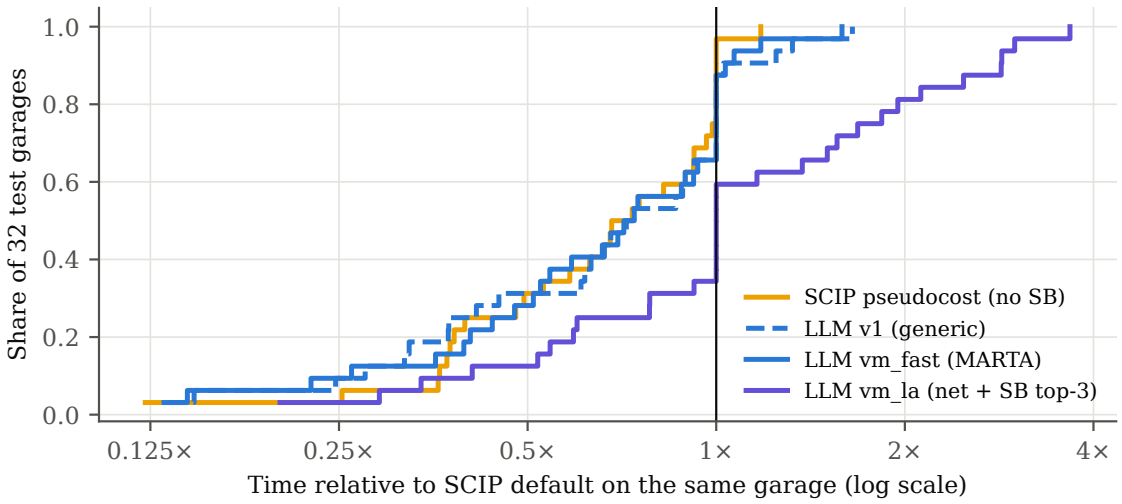


Figure 13: MARTA held-out test (32 garages): cumulative distribution of the per-garage time ratio against SCIP's default in the same batch. Curves to the left are faster. SCIP's plain pseudocost rule and the LLM-written rules overlap.

8 Discussion

What the written weights bought. One clear, attributable gain: independent set in Study 2, where v1 beat both SCIP's default and SCIP's pseudocost rule (Figure 5: pseudocost is 2× slower than the default there, v1 is 2.7× faster; different batches), and beat trained networks with the same inputs. Its terms are interpretable and were written from a few diagnostic reads. Everywhere else, the gain over the default was explained by a cheaper rule that SCIP already ships, or did not exist.

Why. Strong-branching agreement measured from cheap features is low for every feature and every network we tried (0.2–0.46); the structure that makes strong branching valuable in set partitioning (children that can be pruned) is not visible to cheap features. The only route that reached strong-branching-like trees was spending strong branching selectively, and its selection, tuned on 8 practice instances, overfit.

Threats to validity. (i) Runs were concurrent (up to 10 processes on 10 cores); pairing controls load within a batch but absolute times vary across batches. (ii) Studies 2–3 compute features in Python. (iii) Two SCIP versions were used (10.0 via PySCIPOpt, 10.1 natively); the native default solved one practice garage in 21 nodes versus 337 in 10.0, so conclusions are drawn within one build. (iv) Instance sets are small (8–40 per comparison). (v) The MARTA labour rules and garage grouping are synthetic. (vi) Practice instances were chosen as non-trivial for the default. (vii) The trained baseline is small. (viii) The same AI wrote the weights, ran the analysis and wrote this paper; to limit this, every number is produced by a script from logs that are released, and the negative results are reported as found.

9 Conclusion

An LLM can write branching networks that are readable, exact-safe and occasionally better than a mature solver's default on a specific family, and better than gradient-trained weights on the same features. In our application, a real-data bus-driver scheduling model, the measured speed-up of about 37% was real but was entirely reproduced by switching SCIP to its plain pseudocost rule, a one-line configuration change. For practitioners the lesson is to test the cheapest baseline that the solver already offers before attributing a gain to a learned or written rule. For WSP, the open question is whether richer, problem-aware inputs (or a written policy for *when* to spend strong branching) can give the network information that cheap rules lack.

Data and code availability

All code (Rust column generation, Python and C branching rules, MARTA instance generator), network weight files with their notes, raw run logs (JSONL) and the figure script are in the ancillary files of this submission and at `https://github.com/butkeraites/llm-written-weights-or`. Third-party data: XTW instances from CVRPLIB [Uchoa et al., 2026], MIPLIB 2017 [Gleixner et al., 2021], MARTA GTFS [Metropolitan Atlanta Rapid Transit Authority (MARTA), 2026].

References

Tobias Achterberg. *Constraint Integer Programming*. PhD thesis, Technische Universität Berlin, 2007.

Tobias Achterberg. SCIP: solving constraint integer programs. *Mathematical Programming Computation*, 1(1):1–41, 2009. doi: 10.1007/s12532-008-0001-1.

Tobias Achterberg, Thorsten Koch, and Alexander Martin. Branching rules revisited. *Operations Research Letters*, 33(1):42–54, 2005. doi: 10.1016/j.orl.2004.04.002.

Egon Balas and Andrew Ho. Set covering algorithms using cutting planes, heuristics, and subgradient optimization: A computational study. In M. W. Padberg, editor, *Combinatorial Optimization*, volume 12 of *Mathematical Programming Studies*, pages 37–60. Springer Berlin Heidelberg, 1980. doi: 10.1007/BFb0120886.

Roberto Baldacci, Aristide Mingozzi, and Roberto Roberti. New route relaxation and pricing strategies for the vehicle routing problem. *Operations Research*, 59(5):1269–1283, 2011. doi: 10.1287/opre.1110.0975.

Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *Science*, 286 (5439):509–512, 1999. doi: 10.1126/science.286.5439.509.

Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*, 290(2): 405–421, 2021. doi: 10.1016/j.ejor.2020.07.063.

David Bergman, Andre A. Cire, Willem-Jan van Hoeve, and John Hooker. *Decision Diagrams for Optimization*. Artificial Intelligence: Foundations, Theory, and Algorithms. Springer International Publishing, Cham, 2016. doi: 10.1007/978-3-319-42849-9.

Gérard Cornuéjols, Ramaswami Sridharan, and Jean-Michel Thizy. A comparison of heuristics and relaxations for the capacitated plant location problem. *European Journal of Operational Research*, 50(3):280–297, 1991. doi: 10.1016/0377-2217(91)90261-S.

Guy Desaulniers, Jacques Desrosiers, and Marius M. Solomon, editors. *Column Generation*. Springer US, 2005. doi: 10.1007/b135457.

Martin Desrochers, Jacques Desrosiers, and Marius Solomon. A new optimization algorithm for the vehicle routing problem with time windows. *Operations Research*, 40(2):342–354, 1992. doi: 10.1287/opre.40.2.342.

Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL `https://proceedings.neurips.cc/paper/2019/hash/d14c2267d848abeb81fd590f371d39bd-Abstract.html`.

Ambros Gleixner, Gregor Hendel, Gerald Gamrath, Tobias Achterberg, Michael Bastubbe, Timo Berthold, Philipp Christophel, Kati Jarck, Thorsten Koch, Jeff Linderoth, Marco Lübbecke, Hans D. Mittelmann, Derya Ozyurt, Ted K. Ralphs, Domenico Salvagnin, and Yuji Shinano. MIPLIB 2017: data-driven compilation of the 6th mixed-integer programming library. *Mathematical Programming Computation*, 13(3):443–490, 2021. doi: 10.1007/s12532-020-00194-3.

Christopher Hojny, Mathieu Besançon, Ksenia Bestuzheva, Sander Borst, João Dionísio, Johannes Ehls, Leon Eifler, Mohammed Ghannam, Ambros Gleixner, Adrian Göß, Alexander Hoen, Jacob von Holly-Ponientzietz, Rolf van der Hulst, Dominik Kamp, Thorsten Koch, Kevin Kofler, Jurgen Lentz, Marco Lübbecke, Stephen J. Maher, Paul Matti Meinhold, Gioni Mexi, Til Mohr, Erik Mühmer, Krunal Kishor Patel, Marc E. Pfetsch, Sebastian Pokutta, Chantal Reinartz Groba, Felipe Serrano, Yuji Shinano, Mark Turner, Stefan Vigerske, Matthias Walter, Dieter Weninger, and Liding Xu. The SCIP Optimization Suite 10.0. arXiv:2511.18580, 2025. URL `https://arxiv.org/abs/2511.18580`.

Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. doi: 10.1007/s12532-017-0130-5.

Elias B. Khalil, Pierre Le Bodic, Le Song, George L. Nemhauser, and Bistra Dilkina. Learning to branch in mixed integer programming. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, pages 724–731, 2016. doi: 10.1609/aaai.v30i1.10080.

Kevin Leyton-Brown, Mark Pearson, and Yoav Shoham. Towards a universal test suite for combinatorial auction algorithms. In *Proceedings of the 2nd ACM Conference on Electronic Commerce*, pages 66–76. ACM, 2000. doi: 10.1145/352871.352879.

Marco E. Lübbecke and Jacques Desrosiers. Selected topics in column generation. *Operations Research*, 53(6):1007–1023, 2005. doi: 10.1287/opre.1050.0234.

Stephen Maher, Matthias Miltenberger, João Pedro Pedroso, Daniel Rehfeldt, Robert Schwarz, and Felipe Serrano. PySCIPOpt: Mathematical programming in Python with the SCIP Optimization Suite. In Gert-Martin Greuel, Thorsten Koch, Peter Paule, and Andrew Sommese, editors, *Mathematical Software – ICMS 2016*, volume 9725 of *Lecture Notes in Computer Science*, pages 301–307, Cham, 2016. Springer International Publishing. doi: 10.1007/978-3-319-42432-3_37.

Alejandro Marcos Alvarez, Quentin Louveaux, and Louis Wehenkel. A machine learning-based approximation of strong branching. *INFORMS Journal on Computing*, 29(1):185–195, 2017. doi: 10.1287/ijoc.2016.0723.

Metropolitan Atlanta Rapid Transit Authority (MARTA). MARTA GTFS static feed. `https://www.itsmarta.com/app-developer-resources.aspx`, 2026. Feed file `https://itsmarta.com/google_transit_feed/google_transit.zip`, effective 2026-08-22, accessed 2026-11-04.

D. M. Ryan and B. A. Foster. An integer programming approach to scheduling. In Anthony Wren, editor, *Computer Scheduling of Public Transport: Urban Passenger Vehicle and Crew Scheduling*, pages 269–280. North-Holland, 1981.

Rikiya Takehi, Jayden Teoh, Alex Zhang, and Omar Khattab. Sparks of Weight-Space Programming: LLMs can natively read & write neural network weights. `https://rikiyat.github.io/blog/weight-space-programming/`, November 2026. Blog post, MIT CSAIL, published November 2, 2026.

Eduardo Uchoa, Igor Malheiros, Eduardo Queiroga, Anand Subramanian, and Rafael Martinelli. Generation of the XTW and XLTW instances. `https://galgos.inf.puc-rio.br/cvrplib/uploads/files/vrptw_challenge/instances.pdf`, October 2026. CVRPLib VRPTW Challenge, `https://galgos.inf.puc-rio.br/cvrplib/`.

P. Wentges. Weighted Dantzig–Wolfe decomposition for linear mixed-integer programming. *International Transactions in Operational Research*, 4(2):151–162, 1997. doi: 10.1111/j.1475-3995.1997.tb00071.x.

Anthony Wren and Jean-Marc Rousseau. Bus driver scheduling — an overview. In Joachim R. Daduna, Isabel Branco, and José M. Pinto Paixão, editors, *Computer-Aided Transit Scheduling*, volume 430 of *Lecture Notes in Economics and Mathematical Systems*, pages 173–187. Springer Berlin Heidelberg, 1995. doi: 10.1007/978-3-642-57762-8_12.

A Reproducibility

- Synthetic instances: `gen.py OUTDIR N SEED [medium]` (seeds: practice 1, test 2, medium 3, training 4).
- Built-in rules: `run_builtin.py OUT TLIMIT WORKERS RULES INSTANCES`.
- Network rules: `run_net.py OUT TLIMIT WORKERS NETS [-diag K:MAX] INSTANCES`.
- MARTA instances: `marta_gen.py OUTDIR N SEED SERVICE 3` (pool seed 7, test seed 11; services 5, 28, 3, 4).
- Native plugin: `cc -O2 csrc/wspsolve.c -lscip`; runs via `run_c.py OUT 300 10 RULES INSTANCES` with `SCIP_DEFAULTS=1`.
- Figures and every number in the text: `scripts/figs.py`, which writes `figs/summary.json`.

B What the AI did and what the human did

- **Human author:** chose the topic (WSP applied to exact OR), the first target (column generation on XTW), the pivot to LP/MILP, the request for a real application, the port to C, and the decision to publish; approved each step; checked the results.
- **AI (Claude Opus 5.5 in Claude Code):** read the WSP post; designed and wrote all code; wrote every network weight and note; launched all runs on the author's machine; read the diagnostics; wrote the analysis, figures and this text; looked up and verified references.
- **Not done by anyone:** no number was edited by hand; no run was discarded except the instance that SCIP could not read and the superseded Python run of the final test, which was stopped when the C plugin replaced it.

C Claims and evidence

ID	Evidence	Log files (`results/`)	Key in `summary.json`
F1	Fig. 4	`sweep1.jsonl` (wsp-cg)	`cg_*`
F2	Fig. 6, 5	`gen_test`, `gen_medium`, `step0`	`synthetic_*`, `headroom`
F3	Fig. 7, 8	`sb_train`, `vs_trained_*`	`sb_agreement_train`, `hand_vs_trained`
F4	Fig. 9	`miplib`, `miplib2`	`miplib`
F5	Fig. 13, Tab. 2	`c_ctrl`, `c_test`	`marta_test_c`, `paired_tests_time`
F6	Fig. 12, 13	`c_prac`, `c_test`	`marta_practice_c`